Category B Assignment

PROBLEM 1 • Basic

Gym Membership Foundation & Batch Trial Sign-up Validator

A gym wants to move its sign-up process into code. During launch week, applications come in as bulk lists, not one at a time. Build GymMember as the base class with a validated constructor, and PremiumMember as a simple subclass of it. Then write a method that signs up a whole list of member IDs at once and counts how many were rejected, instead of stopping at the first bad one.

Requirements

- PremiumMember must extend GymMember directly (single inheritance) and forward the shared fields via super(...) — do not duplicate memberId or monthlyFee as separate fields in PremiumMember.
- GymMember's constructor must reject a memberId that's blank, whitespace-only, or shorter than 4 characters.
- signUpBatch(...) must attempt to construct one GymMember per array entry, using try/catch to count rejections — it must not pre-validate the strings itself before construction.

Function Signature(s)

```
public GymMember(String memberId, int monthlyFee)
public PremiumMember(String memberId, int monthlyFee, String trainerName)
void attendSession()
int getSessionsAttended()
static String signUpBatch(String[] memberIds, int monthlyFee)
```

Examples

Input	Output
`new GymMember("GM1", 1000)`	`construction rejected`

Explanation: *"GM1" is only 3 characters — one short of the minimum.*

Input	Output

`PremiumMember p = new PremiumMember("MEM01", 2000, "Coach Riya");`
`p.attendSession();`
`p.attendSession();`
`p.getSessionsAttended()`
	`2`

Input	Output
`signUpBatch({"MEM1", "GM1", "MEM2", " ", "MEM3"}, 1000)`	`"Signed Up: 3 | Rejected: 2"`

HINT Let GymMember's constructor be the only place the validation rule lives — signUpBatch should simply try to build one, catch the IllegalArgumentException if construction fails, and count it as a rejection.

CONSTRAINTS: monthlyFee is a positive integer. memberIds.length up to 300.

PROBLEM 2 • Basic → Intermediate

Three Tiers of Gym Membership

The gym is adding two new membership types: an Elite membership with a reserved locker, and a Group Class membership that is completely separate from premium members. Build both classes. Then write a method that identifies which type of membership a given object is, and another method that adds up total sessions attended across a mixed list of memberships.

Requirements

- EliteMember must extend PremiumMember (multilevel inheritance, three classes deep); GroupClassMember must extend GymMember directly, independent of PremiumMember entirely (hierarchical inheritance).
- classifyGeneration(...) must decide its answer using instanceof checks alone — do not add a manual "type" field to any class.
- getTotalSessionsAttended(...) must accept an array holding any mix of the four membership types and sum their attended-session counts using each member's own getSessionsAttended() — it must not check each member's type before adding it in.

Function Signature(s)

```
public EliteMember(String memberId, int monthlyFee, String trainerName, String lockerNumber)
public GroupClassMember(String memberId, int monthlyFee, String className)
static String classifyGeneration(GymMember member)
static int getTotalSessionsAttended(GymMember[] members)
```

Examples

Input	Output

`new GymMember("MEM1", 1000).displayInfo()`
`new PremiumMember("MEM2", 2000, "Coach Riya").displayInfo()`
`new EliteMember("MEM3", 3000, "Coach Arjun", "L12").displayInfo()`
`new GroupClassMember("MEM4", 1500, "Zumba").displayInfo()`
	`"Standard Member | Sessions: 0"`
	`"Premium Member | Trainer: Coach Riya | Sessions: 0"`
	`"Elite Member | Trainer: Coach Arjun | Locker: L12 | Sessions: 0"`
	`"Group Class Member | Class: Zumba | Sessions: 0"`

Input	Output
`classifyGeneration(eliteMember)` `classifyGeneration(groupClassMember)`	`"Multilevel descendant (3 generations deep)"` `"Hierarchical sibling (independent branch)"`

Input	Output
`getTotalSessionsAttended({ premiumMember, eliteMember, groupClassMember })`	9

Explanation: *premiumMember has attended 3 sessions, eliteMember has attended 2, and groupClassMember has attended 4 — 3 + 2 + 4 = 9.*

HINT getTotalSessionsAttended(...) is a one-line loop body once you trust polymorphism — calling getSessionsAttended() on a GymMember reference already runs each object's own version, regardless of which subclass it actually is.

CONSTRAINTS: members.length up to 500. Every subclass overrides displayInfo() to describe itself.

PROBLEM 3 • Intermediate

The Premium Loyalty Discount & Late-Fee Ledger

Members who cancel late are charged a fee, but premium members get a discount on every fee. The gym also wants a record of every fee ever charged to a membership — one that outside code cannot secretly change.

Requirements

- PremiumMember must override chargeLateFee(int amount) using @Override, calling super.chargeLateFee(...) with the amount halved — it must not reimplement the fee logic from scratch.
- Every fee applied (in the base class or any override) must be recorded into a private array field; getLateFeeHistory() must return a defensive copy every time it's called, never the real internal array.

Function Signature(s)

```
protected void chargeLateFee(int amount) // defined in GymMember, overridden in PremiumMember
int[] getLateFeeHistory()
int getTotalLateFees()
```

Examples

Input	Output

`PremiumMember p = new PremiumMember("MEM5", 2000, "Coach Riya");`
`p.chargeLateFee(200);`
`p.getTotalLateFees()`
	`100`

Explanation: *PremiumMember halves every late fee before applying it via super — 200 becomes 100.*

Input	Output

`int[] history = p.getLateFeeHistory();`
`history[0] = 999;`
`p.getLateFeeHistory()`
	`[100]`

Explanation: *The recorded fee is the halved amount (100), not the original 200 passed in — and tampering with the returned array never reaches the membership's real internal history.*

HINT super.chargeLateFee(amount / 2) reuses the parent's own deduction logic and its own array-recording logic in one call — resist writing a second, separate recording step inside PremiumMember.

CONSTRAINTS: Up to 10 late fees per membership. amount is always a positive integer when chargeLateFee is called.

PROBLEM 4 • Intermediate

The Monthly Attendance Announcer

Every month, the gym prints one announcement listing all memberships, mixing standard and premium members together. The announcement should be built as a single piece of text, not many small print statements. For premium memberships specifically, it should also show the trainer's name.

Requirements

- batchPrint(...) must loop over a GymMember[] and call displayInfo() polymorphically — no instanceof-based if-else chain deciding what to print.
- The final announcement must be assembled with StringBuilder, not repeated String concatenation inside the loop.
- For any membership that is genuinely a PremiumMember, use instanceof to guard a downcast that reaches its trainer's name — attempting the same downcast on a membership that isn't a PremiumMember must be avoided, not attempted and caught.

Function Signature(s)

```
static String batchPrint(GymMember[] members)
```

Examples

Input	Output
`batchPrint({ new GymMember("MEM6", 1000), new PremiumMember("MEM7", 2000, "Coach Riya") })`	`"Standard | Sessions: 0 | Premium | Trainer: Coach Riya | Sessions: 0 [Trainer via downcast: Coach Riya] | "`

Input	Output
`GymMember plain = new GymMember("MEM8", 1000); PremiumMember bad = (PremiumMember) plain;`	`ClassCastException at runtime`

Explanation: *This compiles fine — the compiler only checks that PremiumMember is some kind of GymMember — but plain is genuinely not a PremiumMember at runtime, so the cast fails the moment it executes.*

HINT The instanceof check and the downcast go together as a pair — check first, then cast, and only inside the branch where the check already passed. Append to the same StringBuilder across every loop iteration, don't create a new one per membership.

CONSTRAINTS: The array may contain any mix of GymMember and its subclasses, but never null in this problem.

PROBLEM 5 • Intermediate → Advanced

Membership Numbers, Referral Codes & the Weekly Check-in Settlement

The gym wants three things handled together: give every membership a unique ID number that can never be changed after it is created; check if a referral code entered by a member is in the correct format; and go through a week's worth of check-ins, safely skipping any entries that are missing.

Requirements

- Every GymMember must receive a membershipNumber from a shared static counter, incremented once per object, and stored in a final field — never settable from outside and never reassignable after construction.
- isValidReferralCode(...) must check the exact format "G" + two digits + one uppercase letter (e.g. "G45B") using charAt() and Character.isDigit()/isUpperCase() — not a regular expression.
- Provide two overloaded payFee(...) methods: a flat-amount version, and a version that also records a payment mode, reusing the flat version internally rather than duplicating its logic.
- processWeeklyCheckIn(...) must use instanceof to separate GroupClassMember from a regular GymMember, and must never throw on a null batch entry.

Function Signature(s)

```
public GymMember(int monthlyFee) // assigns a final membershipNumber internally
void payFee(int amount)
void payFee(int amount, String mode)
int getFeesPaid()
static boolean isValidReferralCode(String code)
static int getMembersEnrolled()
public GroupClassMember(int monthlyFee, String className)
static String processWeeklyCheckIn(GymMember[] members)
```

Examples

Input	Output

`GymMember m1 = new GymMember(1000);`
`m1.membershipNumber`
`GymMember.getMembersEnrolled()`
	`"GYM-2001"`
	`1`

Input	Output

`isValidReferralCode("G45B")`
`isValidReferralCode("G4B")`
`isValidReferralCode("X45B")`
	`true`
	`false`
	`false`

Explanation: *"G4B" is one character too short; "X45B" has the right shape but the wrong leading letter.*

Input	Output

`m1.payFee(500);`
`m1.payFee(500, "UPI");`
`m1.getFeesPaid()`
	`1000`

Explanation: *Both payments land on the same underlying total — the two-argument overload only records the mode before delegating to the one-argument version.*

Input	Output
`processWeeklyCheckIn({ new GroupClassMember(1500, "Zumba"), null, new GymMember(1000) })`	`"2 processed | 1 null skipped | 1 group | 1 individual"`

HINT Increment the static counter inside the constructor itself, before building the final membershipNumber string from its new value. For the referral code, check the length first — a wrong length should fail immediately, before you ever call charAt() on a position that might not exist.

CONSTRAINTS: Batch size up to 2,000.